

Avaliação de Desempenho de Algoritmos Exatos e Heurísticos para o Problema dos k -Centros

Adriano Araújo Domingos dos Santos¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

R. Cláudio Manoel, 1162 – Savassi - 30.140-100 – Belo Horizonte – MG – Brazil

{adriano.domingos}@sga.pucminas.br

1. Implementação

O grafo é lido de um arquivo em formato de texto pelo `GraphReader`, que extrai o número de vértices n , o número de arestas m e o parâmetro k , seguidos de m linhas, sendo que cada uma representa uma aresta ponderada. As arestas são carregadas utilizando a representação *Forward Star*, a qual é utilizada para executar o algoritmo de Floyd-Warshall. Este algoritmo computa as distâncias mínimas entre todos os pares de vértices e produz uma matriz $n \times n$ de caminhos mínimos. Essa matriz é então fornecida como entrada para cada algoritmo solucionador do problema dos k -centros (Gonzalez, FastMap, WVA-IG e Exact Solver), que a utilizam para selecionar os centros e calcular o raio da solução.

1.1. Floyd-Warshall

O algoritmo de Floyd-Warshall computa as distâncias mínimas entre todos os pares de vértices com complexidade de tempo $O(|V|^3)$. Sua implementação segue a formulação padrão: a matriz de distâncias é inicializada com 0 na diagonal principal e ∞ para os demais pares. Em seguida, as arestas do grafo são inseridas, considerando-se a natureza não-direcionada do grafo. Executam-se então três laços aninhados com a relaxação $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$. A implementação realiza verificações de continuidade para evitar *overflow* e, ao final, atesta a ausência de ciclos negativos. A matriz de distâncias resultante, contendo os caminhos mínimos entre todos os pares, constitui a entrada para todos os algoritmos de k -centros, garantindo que as distâncias satisfaçam a desigualdade triangular.

1.2. Exact Solver

O resolvidor exato opera sobre o problema de decisão associado ao k -centros: dado um raio R , decide-se se existem k centros em V tais que todo vértice esteja a uma distância $\leq R$ de pelo menos um centro. A busca pelo raio mínimo é realizada mediante uma busca binária sobre os valores distintos da matriz de distâncias [Kariv and Hakimi 1979], reduzindo o número de problemas de decisão a $O(\log n^2)$.

Para cada raio candidato R , constroem-se, para cada vértice i , máscaras binárias de 64 bits (*bitsets*) representando o conjunto de vértices j tais que $dist[i][j] \leq R$. O problema de decisão reduz-se a uma instância de *set cover*: existem k centros cujas máscaras, quando combinadas via operação lógica OR (*bitwise*), cobrem todos os n vértices? A exploração exaustiva é realizada por *backtracking*, que tenta todas as combinações de k centros. Para viabilizar a execução em instâncias de porte moderado, quatro técnicas de poda são empregadas:

1. **Eliminação de centros dominados.** Um centro i é dominado por j se o conjunto de cobertura de i está contido no de j . Nesse caso, i é redundante, pois j cobre tudo que i cobriria. Essa dominância é detectada por comparação par-a-par das máscaras, removendo-se permanentemente os centros dominados [Caruso et al. 2003].
2. **Verificação gulosa (*greedy check*).** Antes de iniciar o *backtracking*, aplica-se a heurística gulosa de [Chvátal 1979] para a cobertura de conjuntos: seleciona-se iterativamente o centro que maximiza o número de vértices recém-cobertos. Se a cobertura completa for obtida em até k passos, o raio R é considerado viável e a recursão é evitada completamente, acelerando a verificação.
3. **Ordenação pelo menor número de opções (MRV).** No *backtracking*, seleciona-se o vértice descoberto u com o menor número de centros capazes de cobri-lo (heurística *minimum remaining values* [Haralick and Elliott 1980]). Isso minimiza o fator de ramificação nos níveis iniciais da árvore, podendo rapidamente regiões inviáveis do espaço de busca.
4. **Ordenação dos candidatos por cobertura.** Dentro da lista de centros que cobrem o vértice u , os candidatos são ordenados em ordem decrescente pelo tamanho do seu conjunto de cobertura. Centros que cobrem mais vértices são testados primeiro, aumentando a probabilidade de encontrar uma cobertura viável cedo na busca.

O algoritmo retorna a solução ótima exata para o raio mínimo, correspondente ao menor R para o qual o problema de decisão se prova viável.

1.3. Gonzalez

O algoritmo de [Gonzalez 1985] é uma heurística construtiva que oferece uma 2-aproximação para o problema dos k -centros. A execução inicia selecionando arbitrariamente o primeiro vértice (índice 0) como primeiro centro. Em cada iteração subsequente, o vértice mais distante do conjunto de centros já selecionados é escolhido como o próximo centro, repetindo-se esse processo até que k centros sejam selecionados. Sua complexidade é $O(nk)$, sendo extremamente rápido na prática. Seu comportamento é determinístico, produzindo sempre a mesma solução para uma dada instância.

1.4. FastMap

O algoritmo FastMap [Faloutsos and Lin 1995] projeta os vértices do grafo em um espaço euclidiano bidimensional, preservando aproximadamente as distâncias originais. A projeção é realizada selecionando-se dois vértices pivôs (os mais distantes entre si) e projetando cada vértice sobre a reta que os conecta. Após a projeção em 2D, o algoritmo aplica a inicialização k -means++ para selecionar os k centros no espaço projetado. O k -means++ seleciona o primeiro centro aleatoriamente e os demais com probabilidade proporcional ao quadrado da distância ao centro mais próximo. Trata-se de um algoritmo aleatorizado, sendo necessária a utilização de uma semente (*seed*) para garantir reprodutibilidade.

1.5. WVA-IG

WVA-IG (*Worst Vertex Adjacent - Iterated Greedy*) é uma metaheurística que combina uma solução inicial obtida pelo algoritmo de Gonzalez com um processo iterativo de

refinamento. Em cada iteração, identifica-se o vértice mais distante de seu centro mais próximo (v^* , o *pior* vértice), adiciona-se v^* como centro temporário (totalizando $k + 1$ centros) e remove-se o centro cuja exclusão resulte no menor aumento de raio. Em seguida, aplica-se uma busca local por trocas (*swap*), testando a substituição de cada centro por um vértice não-centro. Se o algoritmo estagnar por 3 iterações consecutivas sem melhorias, aplica-se uma perturbação aleatória (substituição de centros escolhidos ao acaso). O algoritmo executa até 200 iterações e retorna a melhor solução encontrada.

2. Metodologia

Os experimentos foram realizados utilizando as 40 instâncias da OR-Library [Beasley 1990] para o problema das p -medianas, adaptadas para o problema dos k -centros. Essas instâncias possuem um número de vértices n variando de 100 a 900, e valores de k proporcionais ao tamanho da instância.

Ressalta-se que as instâncias da OR-Library contêm arestas paralelas com pesos distintos entre um mesmo par de vértices, introduzindo ambiguidade no cálculo das distâncias mínimas. Foram avaliadas quatro estratégias de tratamento: sobrescrita do peso pela aresta mais recente; manutenção do menor peso entre as arestas paralelas; manutenção apenas da primeira aresta encontrada (ignorando as paralelas); e armazenamento de pesos direcionais distintos para cada sentido. Como nenhuma dessas abordagens produziu resultados padronizados para todas as instâncias em relação aos *benchmarks* da literatura, os experimentos foram conduzidos ignorando as arestas paralelas: a primeira aresta lida para cada par é mantida, e as demais são descartadas. Essa particularidade deve ser considerada na interpretação dos resultados, especialmente para o resolvedor exato.

Cada algoritmo foi executado 5 vezes para cada instância, totalizando 200 execuções por algoritmo. Foi estipulado um tempo limite de 1 hora para a execução de cada rotina, limite este motivado pelo custo computacional inerente ao *Exact Solver*.

2.1. Métricas de Avaliação

As métricas utilizadas para a avaliação dos algoritmos foram: (i) **raio** obtido, definido como a maior distância de um vértice ao centro mais próximo; (ii) **erro percentual** em relação ao raio de referência; e (iii) **tempo de execução** do algoritmo em milissegundos, excluindo o tempo de pré-processamento do Floyd-Warshall.

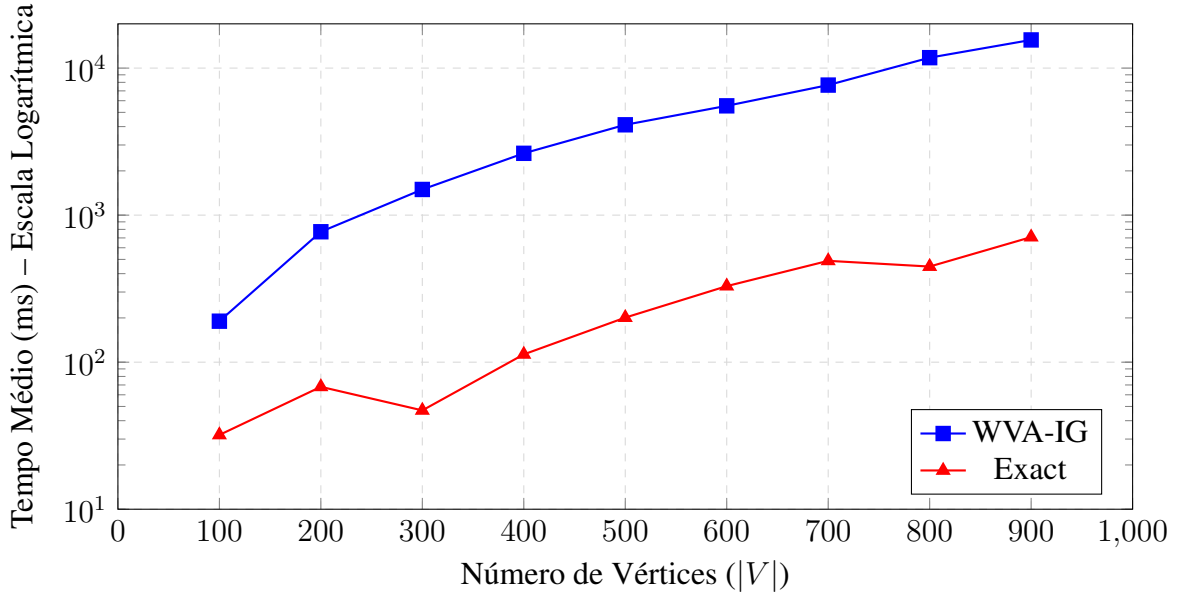
3. Resultados

Os experimentos foram conduzidos conforme a metodologia descrita na Seção 2. As Tabelas 1 e 2 apresentam as médias entre as 5 execuções de cada uma das 40 instâncias da OR-Library.

Os resultados mostram que o algoritmo de Gonzalez obteve um erro médio de 49,84% em relação ao raio de referência, sendo o mais rápido juntamente com o FastMap (menos de 1 ms). O FastMap apresentou erros maiores, com média de 95,30%, sendo superado pelo Gonzalez na grande maioria dos casos. O WVA-IG destacou-se como a heurística de melhor qualidade, com erro médio de 13,17%, frequentemente encontrando soluções ótimas ou muito próximas do ótimo. O resolvedor exato não conseguiu finalizar a execução em todas as 40 instâncias; apenas 29 foram concluídas dentro do tempo limite de 1 hora. Nas instâncias resolvidas, obteve um erro médio de $-0,20\%$, variação

que é explicada pelas inconsistências de modelagem das arestas paralelas das instâncias, conforme discutido na Seção 2.

Figura 1. Tempo médio de execução em função do número de vértices, para instâncias com $k = 5$. Eixo y em escala logarítmica.

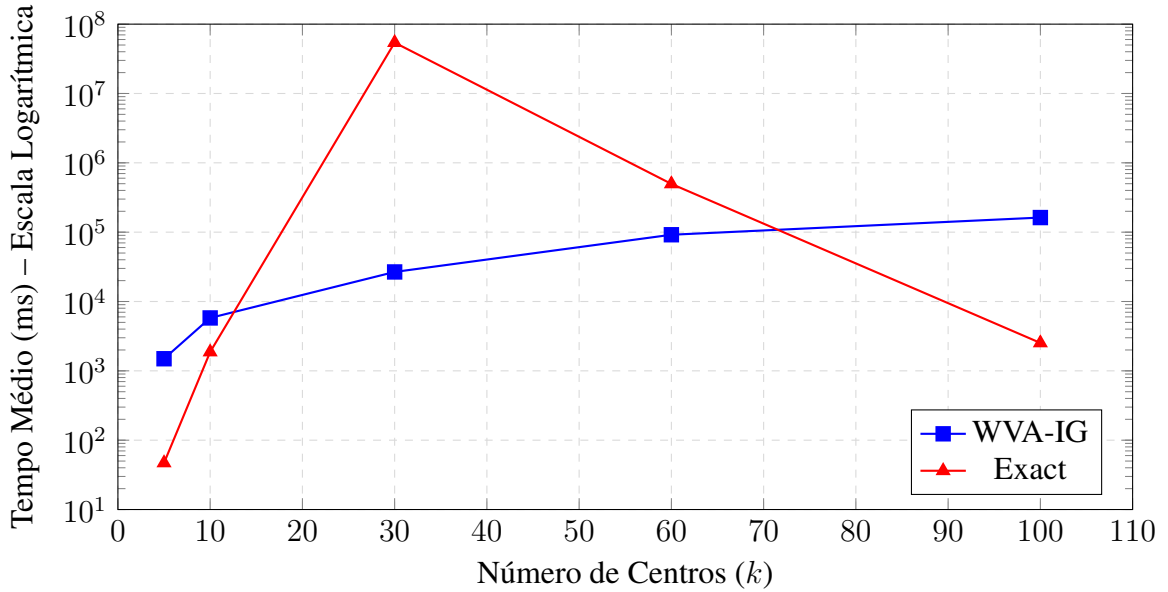


Ressalta-se que o resolvidor exato foi executado com um tempo limite de 1 hora por instância. A instância *pmed13* ($|V| = 300$, $k = 30$) foi incluída nos resultados para demonstrar a escalabilidade do custo computacional, tendo levado aproximadamente 15 horas (54.150.055 ms) para encontrar a solução de forma isolada. Diferentemente das demais instâncias não concluídas, que foram abortadas no limite de 1 hora, essa execução foi permitida até o fim para fins de análise. Por esse motivo, seu tempo de execução integra as tabelas e figuras, mas deve ser interpretado como um caso atípico de extrapolação do limite prático adotado no experimento.

A Figura 1 apresenta o tempo médio de execução em função de $|V|$ para instâncias com $k = 5$. O WVA-IG varia de 190 ms ($|V| = 100$) a 15,5 s ($|V| = 900$), enquanto o resolvidor exato mantém-se abaixo de 1 s em todos os cenários de baixo k .

A Figura 2 ilustra o comportamento em função de k para $n = 300$. O WVA-IG apresenta crescimento monotônico de tempo em relação a k , variando de 1,5 s ($k = 5$) a 162 s ($k = 100$). O resolvidor exato, por outro lado, exibe um comportamento não-monotônico: é extremamente rápido para k pequeno (47 ms em $k = 5$) e para k elevado (2,5 s em $k = 100$), contudo, torna-se drasticamente mais lento em valores intermediários de k , atingindo 15 horas em $k = 30$ e 8 minutos em $k = 60$. Esse fenômeno é uma consequência direta da natureza do problema de *set cover* subjacente: quando k é restrito, a árvore de busca é rasa; quando k é permissivo, a heurística gulosa (*greedy check*) encontra uma cobertura viável quase instantaneamente. O pior caso ocorre em valores intermediários de k , onde a profundidade moderada da recursão, aliada à falha da heurística gulosa, força o algoritmo a explorar exaustivamente um espaço de busca colossal. Nos demais cenários abordados, os algoritmos Gonzalez e FastMap executam em menos de 1

Figura 2. Tempo médio de execução em função do número de centros k , para instâncias com $n = 300$. Eixo y em escala logarítmica.



ms, confirmando sua adequação para aplicações onde a velocidade absoluta é imperativa.

4. Conclusão

Este trabalho apresentou a implementação e a comparação de quatro abordagens para o problema dos k -centros em grafos não-direcionados: Gonzalez (2-aproximação determinística), FastMap (projeção 2D associada ao k -means++), WVA-IG (metaheurística iterativa) e um resolvidor exato (busca binária ancorada em *set cover*). Os experimentos utilizando as 40 instâncias da OR-Library demonstraram que o WVA-IG oferece o melhor compromisso entre qualidade de solução e tempo de processamento para instâncias de grande porte, frequentemente alcançando resultados ótimos. O algoritmo de Gonzalez, apesar de extremamente veloz, produz soluções com erro médio em torno de 50%, sendo recomendado apenas para sistemas que exigem latência mínima e toleram aproximações. O FastMap não se mostrou competitivo neste domínio específico, exibindo alta variabilidade e resultados inferiores ao Gonzalez. Por fim, o resolvidor exato demonstra alta eficácia e eficiência para instâncias pequenas ou cenários extremos de k ($|V| \leq 300$), porém sofre com a explosão combinatória em valores intermediários, inviabilizando seu uso irrestrito em instâncias mais complexas.

Referências

- Beasley, J. E. (1990). Or-library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072.
- Caruso, C., Colorni, A., and Aloï, L. (2003). Dominant, an algorithm for the p -center problem. *European Journal of Operational Research*, 149(1):53–64.
- Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235.

- Faloutsos, C. and Lin, K.-I. (1995). Fastmap: A fast algorithm for indexing, data-mining and visualization of traditional and multimedia datasets. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, pages 163–174.
- Gonzalez, T. F. (1985). Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306.
- Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313.
- Kariv, O. and Hakimi, S. L. (1979). An algorithmic approach to network location problems. I: The p -centers. *SIAM Journal on Applied Mathematics*, 37(3):513–538.

Tabela 1. Raio encontrado para as 40 instâncias da OR-Library.
(Para algoritmos não-exatos, os valores correspondem à média das 5 tentativas)

Inst.	n	k	Ótimo	Gonzalez	FastMap	WVA-IG	Exact
pmed1	100	5	127	186 (46,46%)	175,60 (38,27%)	127,20 (0,16%)	127 (0,00%)
pmed2	100	10	98	131 (33,67%)	152,80 (55,92%)	98 (0,00%)	98 (0,00%)
pmed3	100	10	93	154 (65,59%)	145,80 (56,77%)	96,20 (3,44%)	94 (1,08%)
pmed4	100	20	74	112 (51,35%)	139,20 (88,11%)	76 (2,70%)	74 (0,00%)
pmed5	100	33	48	72 (50,00%)	94,40 (96,67%)	48 (0,00%)	48 (0,00%)
pmed6	200	5	84	138 (64,29%)	115 (36,90%)	82,60 (-1,67%)	82 (-2,38%)
pmed7	200	10	64	98 (53,13%)	94,60 (47,81%)	65,20 (1,88%)	64 (0,00%)
pmed8	200	20	55	81 (47,27%)	101,80 (85,09%)	58,60 (6,55%)	55 (0,00%)
pmed9	200	40	37	61 (64,86%)	77,60 (109,73%)	40 (8,11%)	39 (5,41%)
pmed10	200	67	20	30 (50,00%)	59,20 (196,00%)	21,20 (6,00%)	20 (0,00%)
pmed11	300	5	59	74 (25,42%)	84,40 (43,05%)	59,40 (0,68%)	57 (-3,39%)
pmed12	300	10	51	71 (39,22%)	87 (70,59%)	52,20 (2,35%)	51 (0,00%)
pmed13	300	30	35	60 (71,43%)	67,20 (92,00%)	38,80 (10,86%)	36 (2,86%)
pmed14	300	60	26	40 (53,85%)	61,40 (136,15%)	30,60 (17,69%)	25 (-3,85%)
pmed15	300	100	18	25 (38,89%)	52,80 (193,33%)	20 (11,11%)	17 (-5,56%)
pmed16	400	5	47	84 (78,72%)	64,80 (37,87%)	45 (-4,26%)	45 (-4,26%)
pmed17	400	10	39	56 (43,59%)	59 (51,28%)	41,60 (6,66%)	39 (0,00%)
pmed18	400	40	28	45 (60,71%)	54,20 (93,57%)	32,80 (17,15%)	—
pmed19	400	80	18	29 (61,11%)	43 (138,89%)	23,40 (30,00%)	—
pmed20	400	133	13	19 (46,15%)	38,80 (198,46%)	16,60 (27,69%)	13 (0,00%)
pmed21	500	5	40	53 (32,50%)	54,60 (36,50%)	40 (0,00%)	40 (0,00%)
pmed22	500	10	38	56 (47,37%)	51,60 (35,79%)	39,60 (4,21%)	38 (0,00%)
pmed23	500	50	22	35 (59,09%)	44,80 (103,64%)	27,60 (25,45%)	—
pmed24	500	100	15	23 (53,33%)	33,80 (125,33%)	20 (33,33%)	—
pmed25	500	167	11	15 (36,36%)	36,40 (230,91%)	14,20 (29,09%)	11 (0,00%)
pmed26	600	5	38	50 (31,58%)	51,60 (35,79%)	38,60 (1,58%)	38 (0,00%)
pmed27	600	10	32	43 (34,38%)	44,20 (38,13%)	34 (6,25%)	32 (0,00%)
pmed28	600	60	18	28 (55,56%)	35,80 (98,89%)	22,40 (24,44%)	—
pmed29	600	120	13	19 (46,15%)	33 (153,84%)	17 (30,77%)	—
pmed30	600	200	9	14 (55,56%)	28 (211,11%)	13,20 (46,66%)	—
pmed31	700	5	30	43 (43,33%)	40,80 (36,00%)	30 (0,00%)	30 (0,00%)
pmed32	700	10	29	45 (55,17%)	65,60 (126,21%)	30,60 (5,52%)	29 (0,00%)
pmed33	700	70	15	25 (66,67%)	31,20 (108,00%)	20,80 (38,67%)	—
pmed34	700	140	11	17 (54,55%)	29,80 (170,91%)	15,40 (40,00%)	—
pmed35	800	5	30	38 (26,67%)	42 (40,00%)	30 (0,00%)	30 (0,00%)
pmed36	800	10	27	41 (51,85%)	50,80 (88,15%)	29 (7,41%)	27 (0,00%)
pmed37	800	80	15	24 (60,00%)	31 (106,67%)	20,40 (36,00%)	—
pmed38	900	5	29	36 (24,14%)	45,20 (55,86%)	29 (0,00%)	29 (0,00%)
pmed39	900	10	23	35 (52,17%)	33,80 (46,96%)	25 (8,70%)	24 (4,35%)
pmed40	900	90	13	21 (61,54%)	25,60 (96,92%)	18,40 (41,54%)	—

Tabela 2. Tempo médio de execução (ms) por instância. Para algoritmos não-determinísticos (FastMap e WVA-IG), os valores correspondem à média das 5 tentativas.

Inst.	n	k	Gonzalez	FastMap	WVA-IG	Exact
pmed1	100	5	< 1	< 1	190	32
pmed2	100	10	< 1	< 1	741	99
pmed3	100	10	< 1	< 1	529	131
pmed4	100	20	< 1	< 1	1.537	12
pmed5	100	33	< 1	< 1	3.122	2
pmed6	200	5	< 1	< 1	771	68
pmed7	200	10	< 1	< 1	2.116	2.443
pmed8	200	20	< 1	< 1	6.136	88.451
pmed9	200	40	< 1	< 1	24.318	15.066
pmed10	200	67	< 1	2	43.228	5
pmed11	300	5	< 1	< 1	1.495	47
pmed12	300	10	< 1	< 1	5.793	1.870
pmed13	300	30	< 1	< 1	26.687	54.150.055
pmed14	300	60	< 1	< 1	91.777	496.506
pmed15	300	100	< 1	< 1	162.394	2.527
pmed16	400	5	< 1	< 1	2.633	113
pmed17	400	10	< 1	< 1	7.952	12.701
pmed18	400	40	< 1	< 1	92.161	—
pmed19	400	80	< 1	< 1	257.374	—
pmed20	400	133	< 1	< 1	480.379	798
pmed21	500	5	< 1	< 1	4.115	201
pmed22	500	10	< 1	< 1	11.011	70.691
pmed23	500	50	< 1	< 1	194.555	—
pmed24	500	100	< 1	< 1	555.325	—
pmed25	500	167	< 1	< 1	1.075.321	662.118
pmed26	600	5	< 1	< 1	5.538	329
pmed27	600	10	< 1	< 1	22.621	414.457
pmed28	600	60	< 1	< 1	367.121	—
pmed29	600	120	< 1	< 1	1.008.528	—
pmed30	600	200	< 1	< 1	1.915.908	—
pmed31	700	5	< 1	< 1	7.665	489
pmed32	700	10	< 1	< 1	29.876	509.250
pmed33	700	70	< 1	< 1	759.810	—
pmed34	700	140	< 1	< 1	1.983.417	—
pmed35	800	5	< 1	< 1	11.775	447
pmed36	800	10	< 1	< 1	35.975	792.275
pmed37	800	80	< 1	< 1	1.219.323	—
pmed38	900	5	< 1	< 1	15.543	709
pmed39	900	10	< 1	< 1	43.406	256.296
pmed40	900	90	< 1	< 1	2.133.335	—